

Pulsar

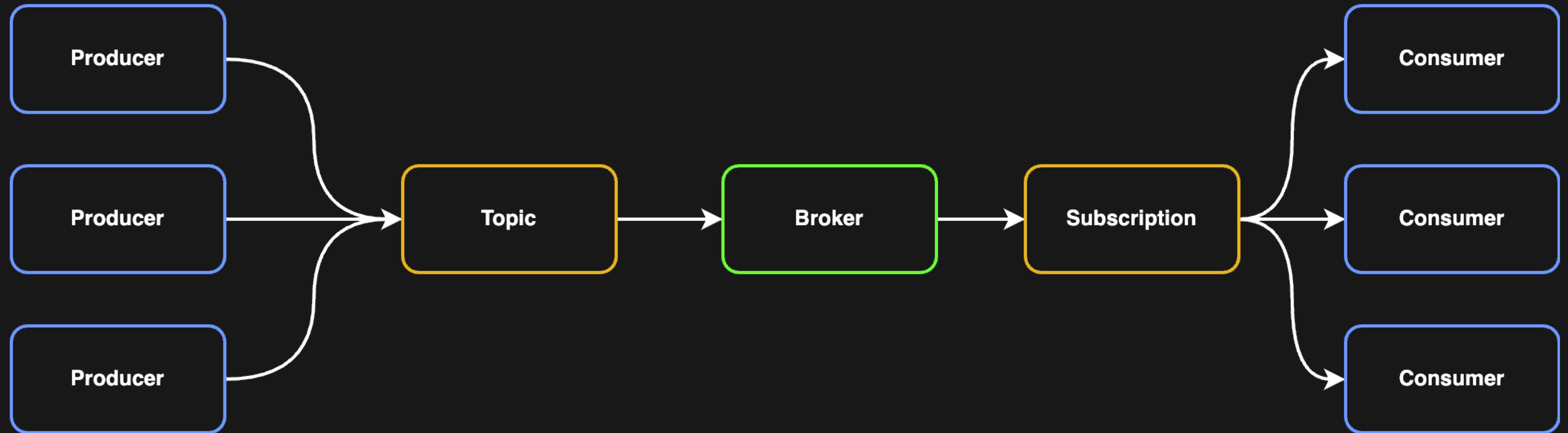
Best Practices

Table of Content

- Topic and partitions
- Producer
- Consumer
- Tools

Concept

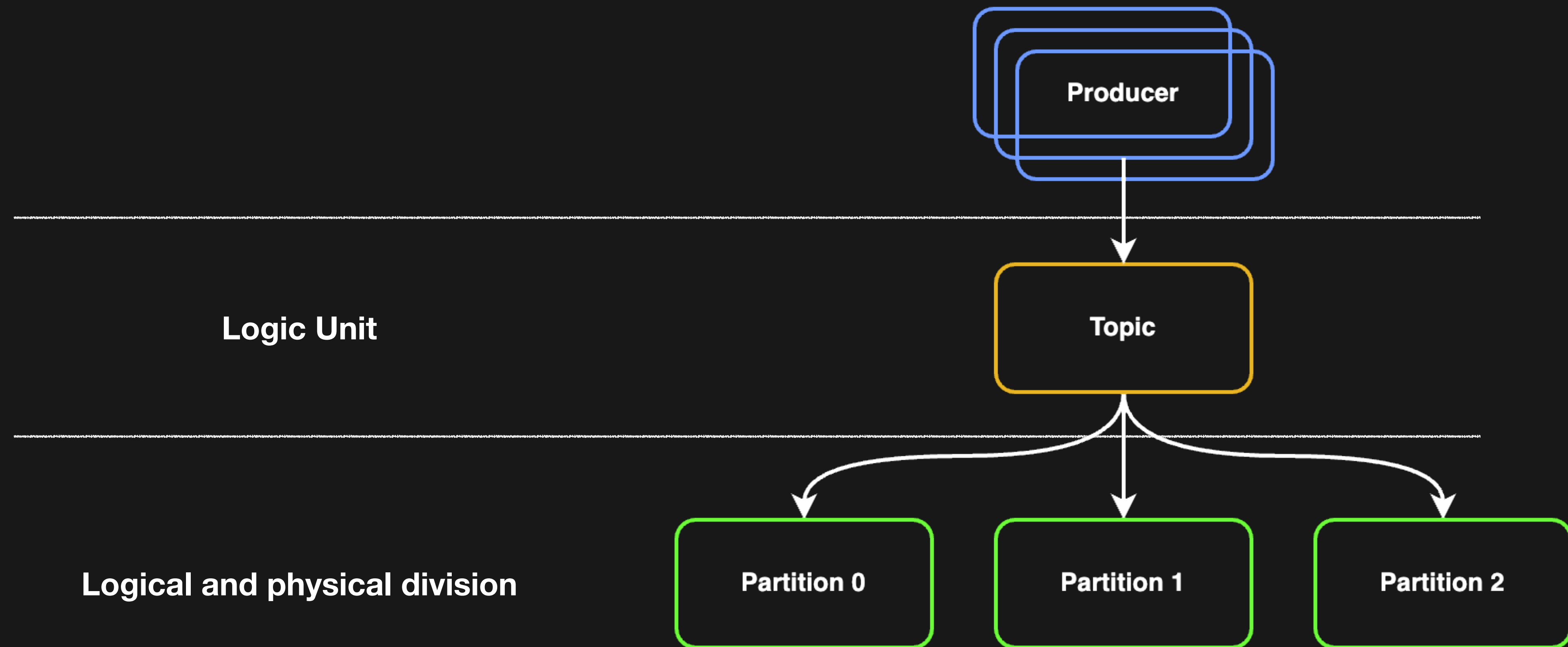
Pub-Sub Model



Topic

- Persistent:
 - Partitioned topic
 - Non-partitioned topic
- Non-Persistent Topic

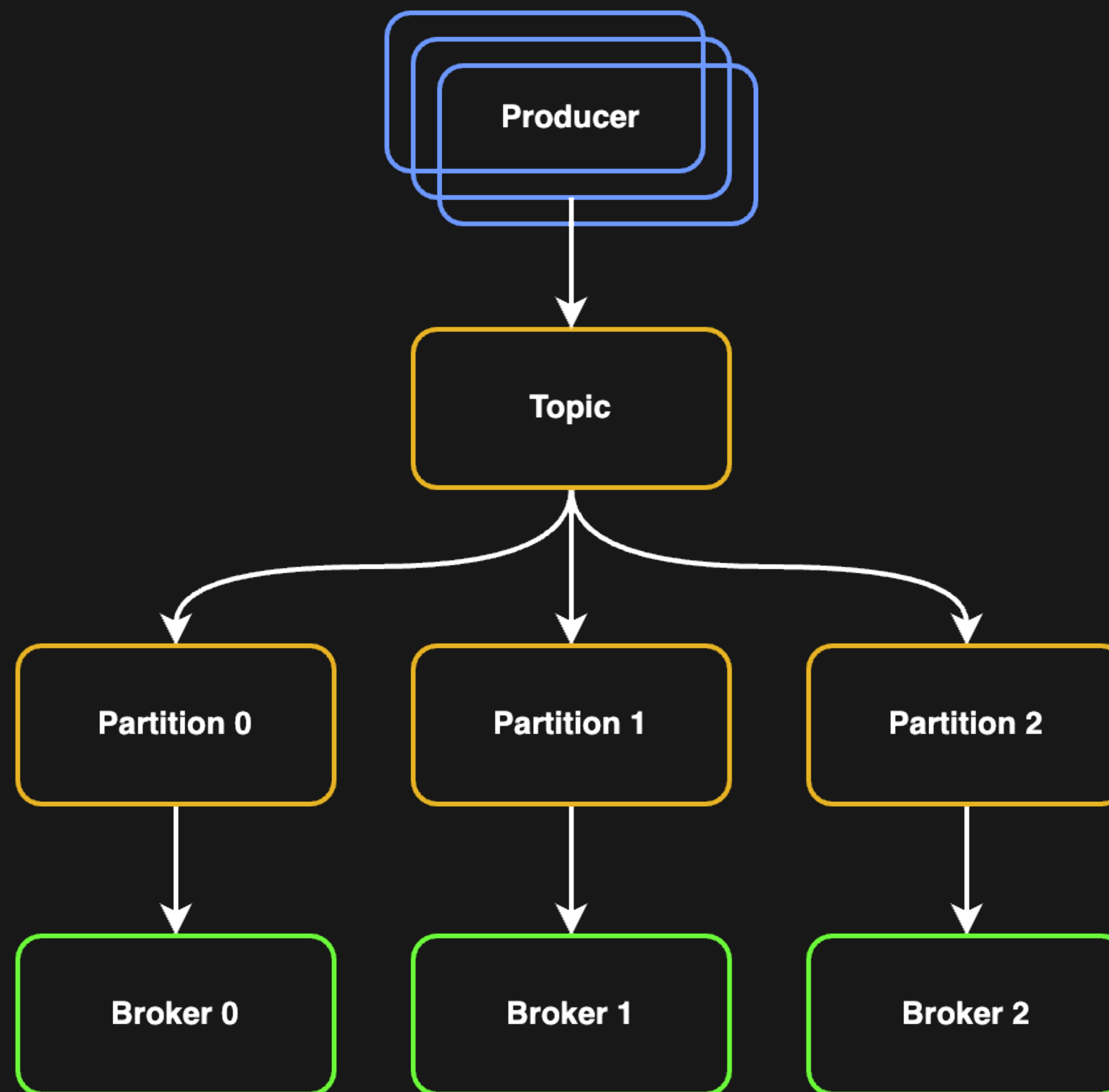
Partitioned Topic



Partitioned Topic

- Consumer:
 - Periodically fetch partition info
 - For Go SDK, each partition creates several (2~3) go routines

Partition Assignment Bundles



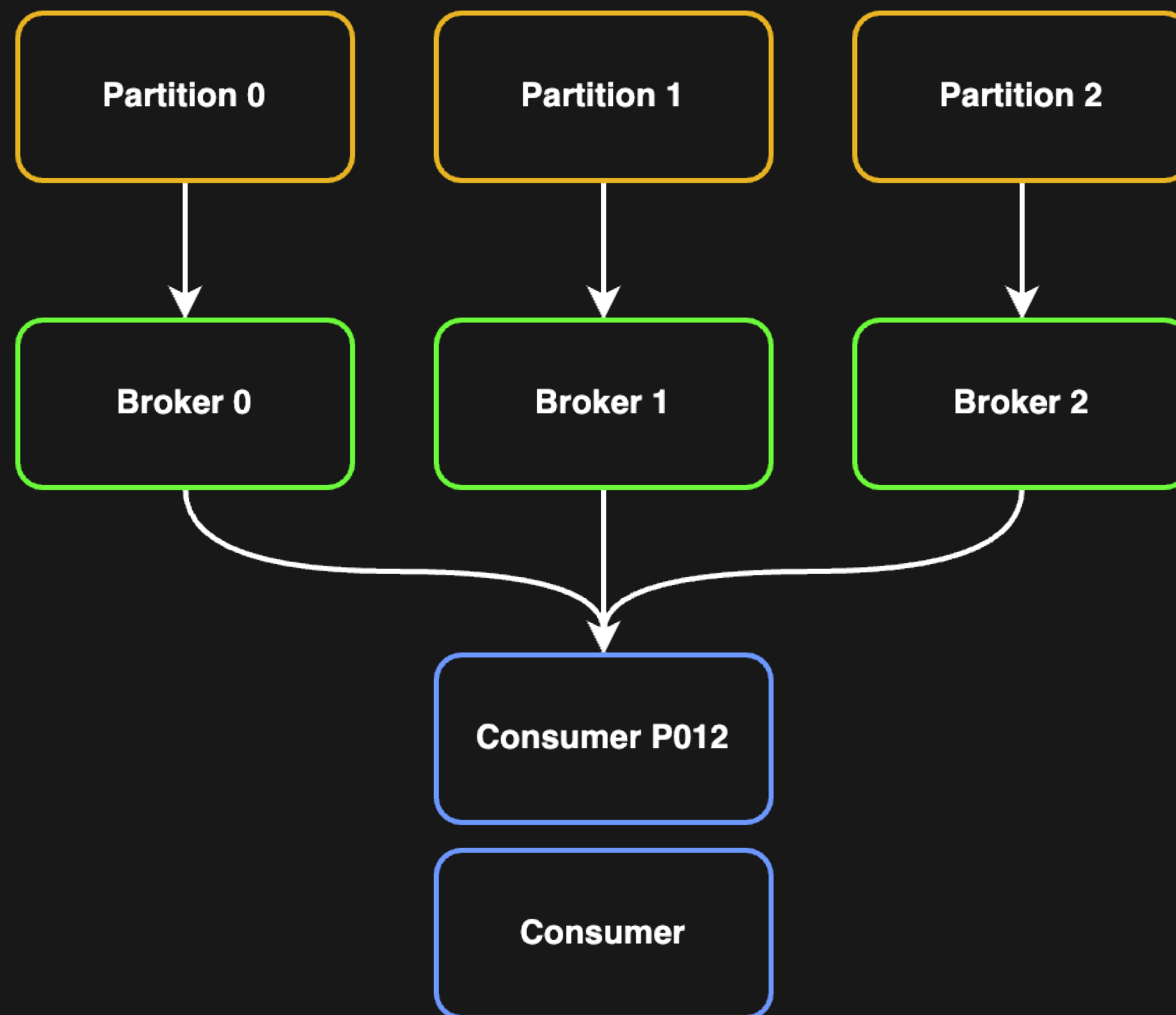
Bundle

- Each bundle comprising a portion of the overall hash range of the namespace
- Topics (partitions) are assigned to a particular bundle
- Dynamic assignment

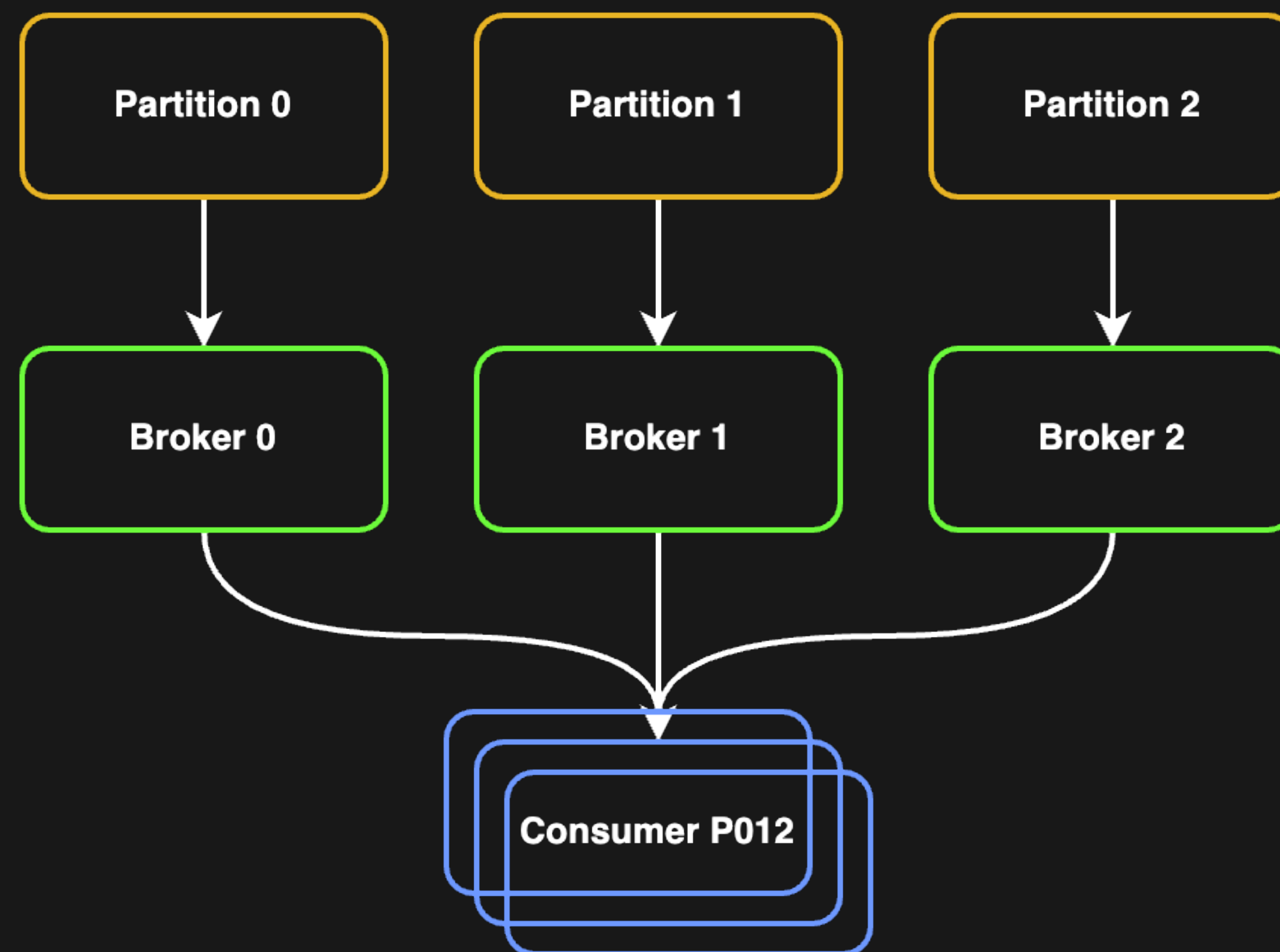
Best Practices #1

- Increase the number of partitions to achieve:
 - Higher throughput
 - Load balancing

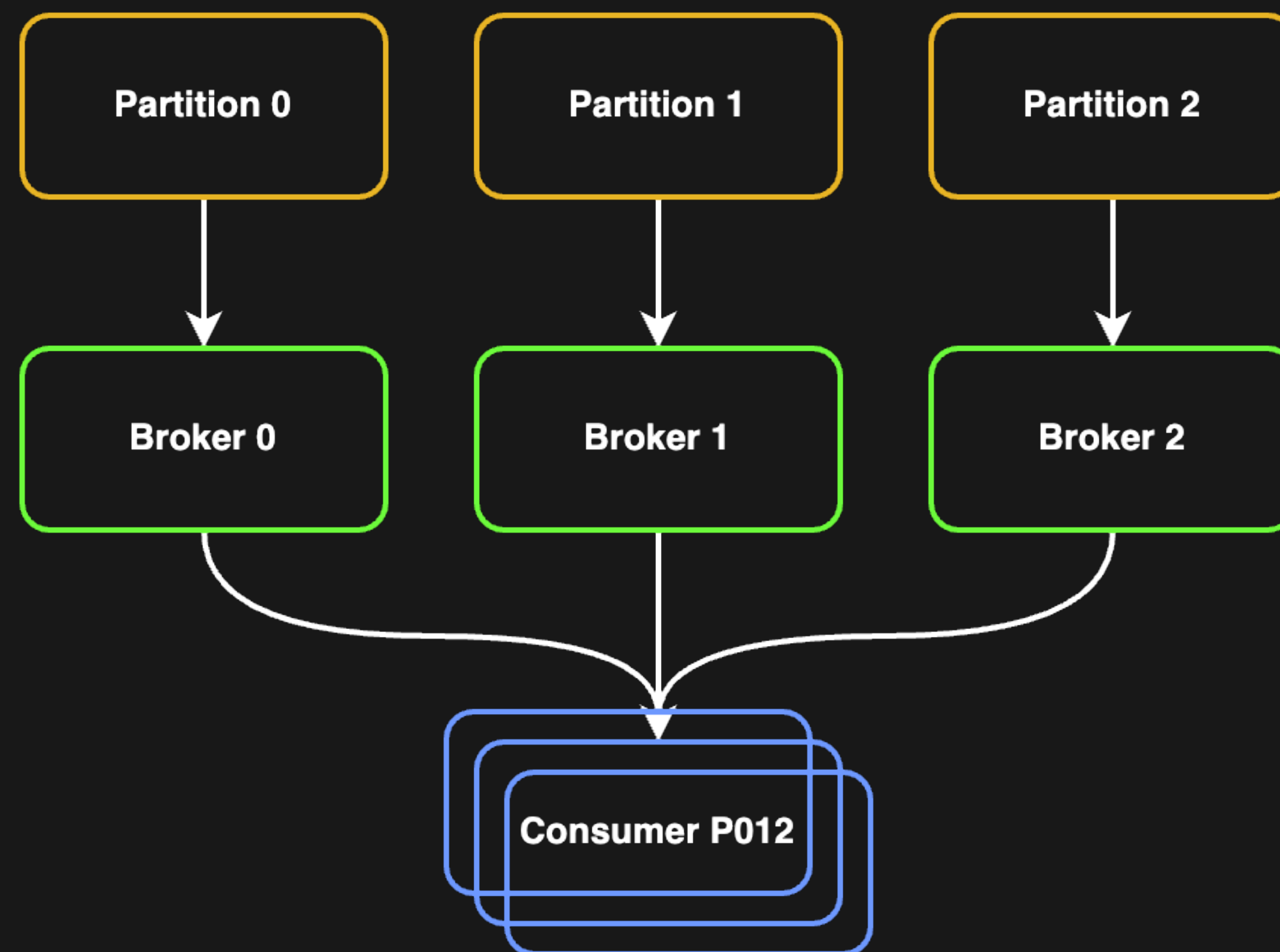
Subscription Failover/Exclusive Mode



Subscription Shared Mode



Subscription Key-Shared Mode



Messages with same key are consistently routed to the same con

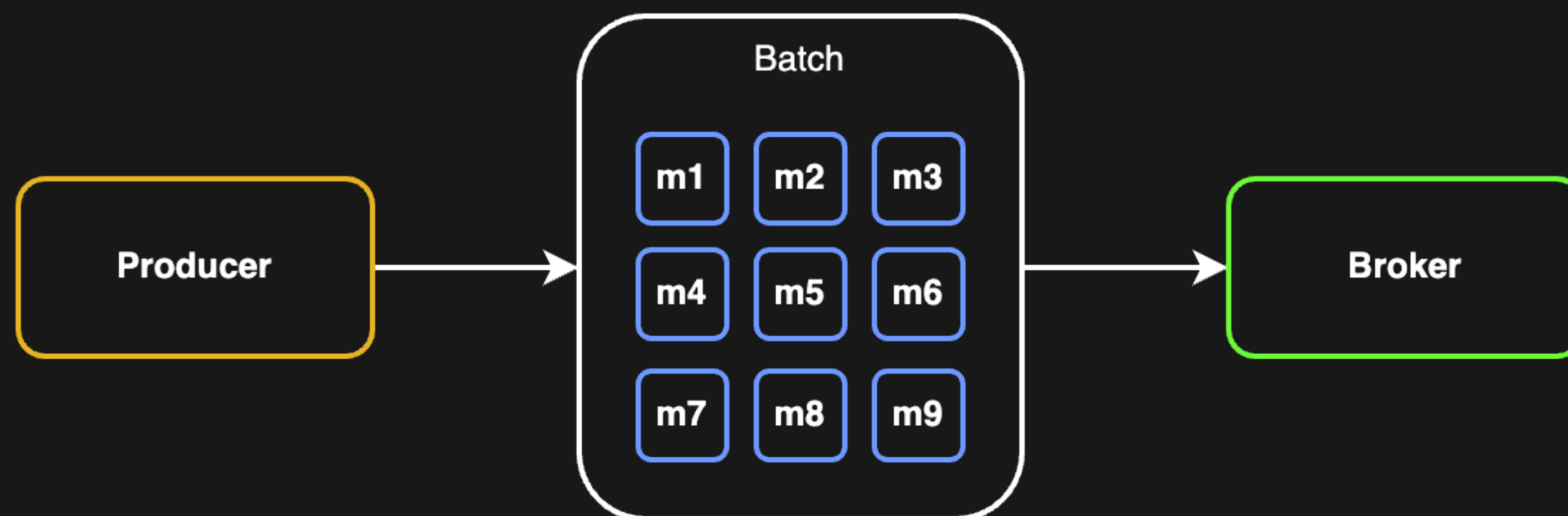
Best Practices #2

- Use Shared Subscription to increase throughput
- Use KeyShared Subscription if ordering is crucial
 - Has constraints

Batching



Single



Batching

Benefits of Batching

- Improve publish throughput
- Improve network
 - Add compression
 - Reduce RPC
- Reduce broker CPU, IO and storage

```
opt = &pulsar.ProducerOption{
    ProducerOption: mq.ProducerOption{
        ConnectionOption: mq.ConnectionOption{},
        Topic: topic,
        Name: "producer-name",
        DisableBatching: false,
        MaxPendingMessages: 100,
        MaxBatchPublishDelay: time.Second,
        MaxBatchingMessages: 100,
        MaxBatchingSize: 1024 * 1024,
    },
}
```

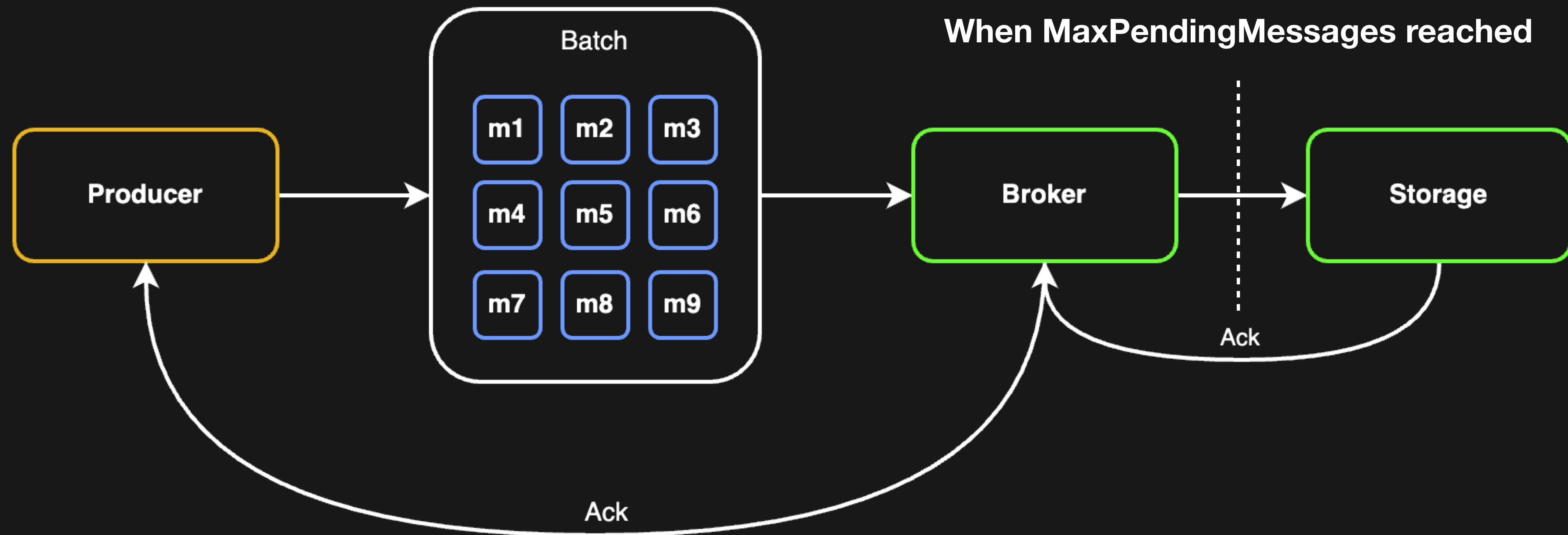
Pitfalls of Batching

- MUST NOT use with KeyShared
- MUST NOT use with Delayed Messages
- A batch is acknowledged only when all messages in a batch are acknowledged.
 - Can result in a redelivery of all messages

Best Practices #3

- Use batching to increase producer throughput only when you know the pitfalls
- In go.planetmeican.com/nerds/mq, batching is disabled by default
- Pay attention to `MaxBatchingSize`

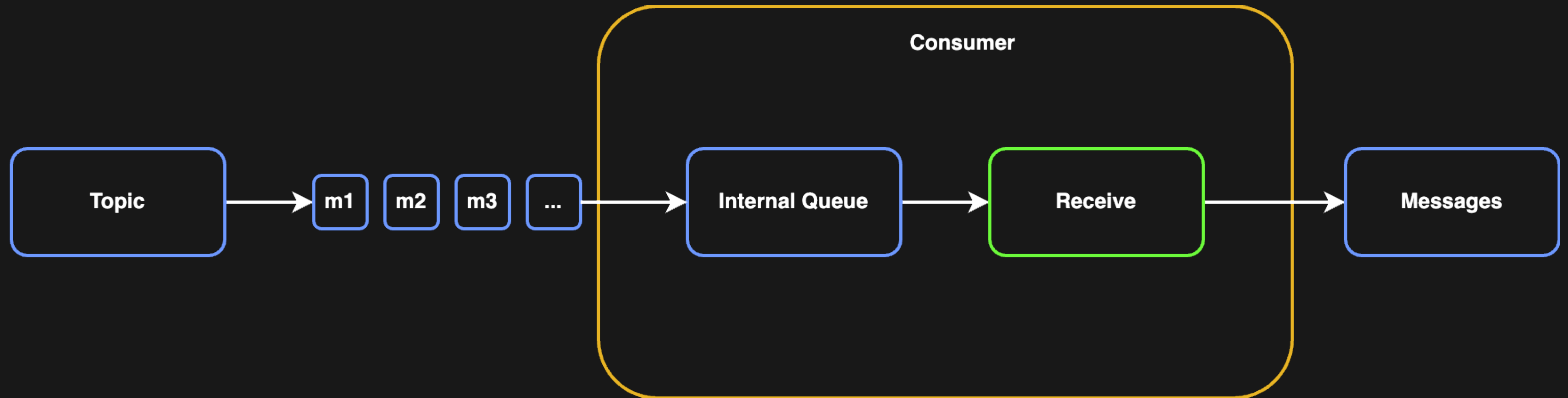
Max Pending Messages



Best Practices #4

- Increase MaxPendingMessages to increase publish throughput
- Increased memory consumption

Consumer Working Principals



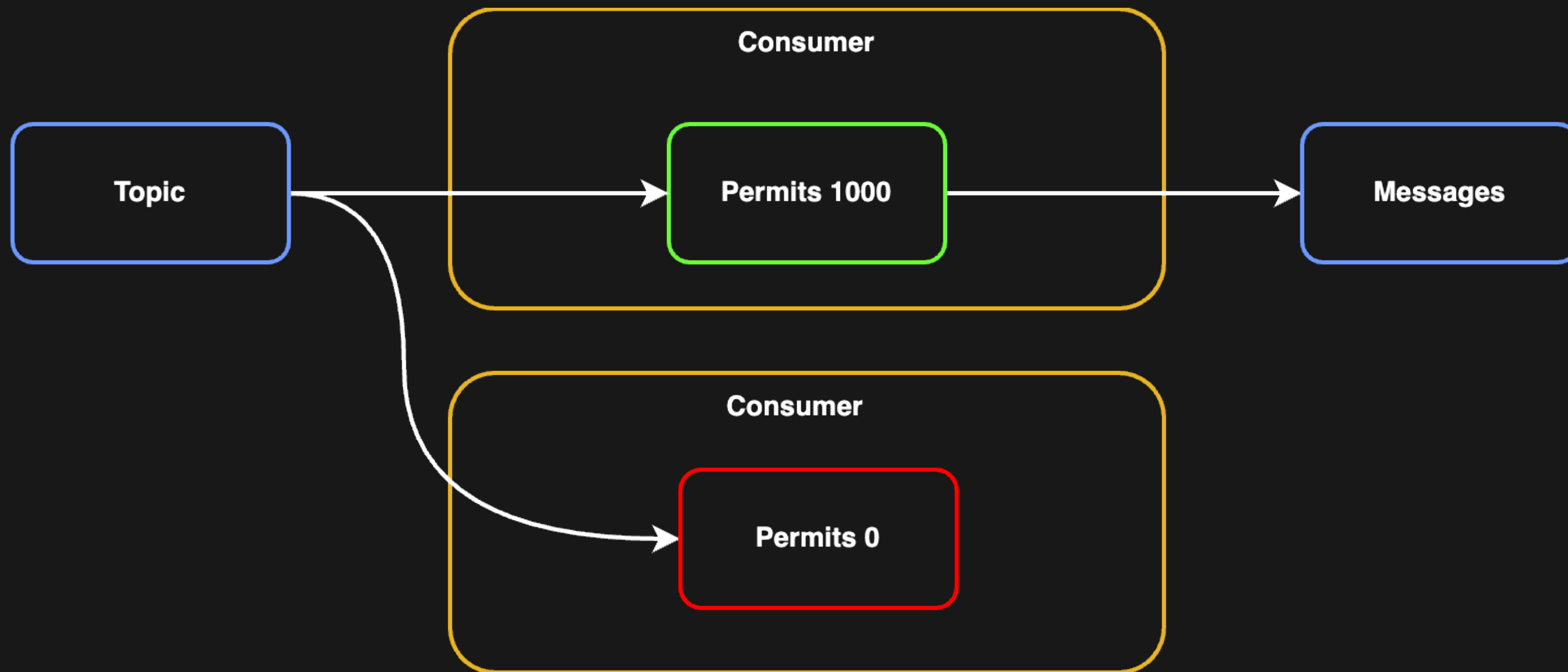
Best Practices #5

- Increasing incoming queue size can improve the throughput
- Default is 1000
- For “Producer.DeliverAt”, set “ReceiverQueueSize” to 0

```

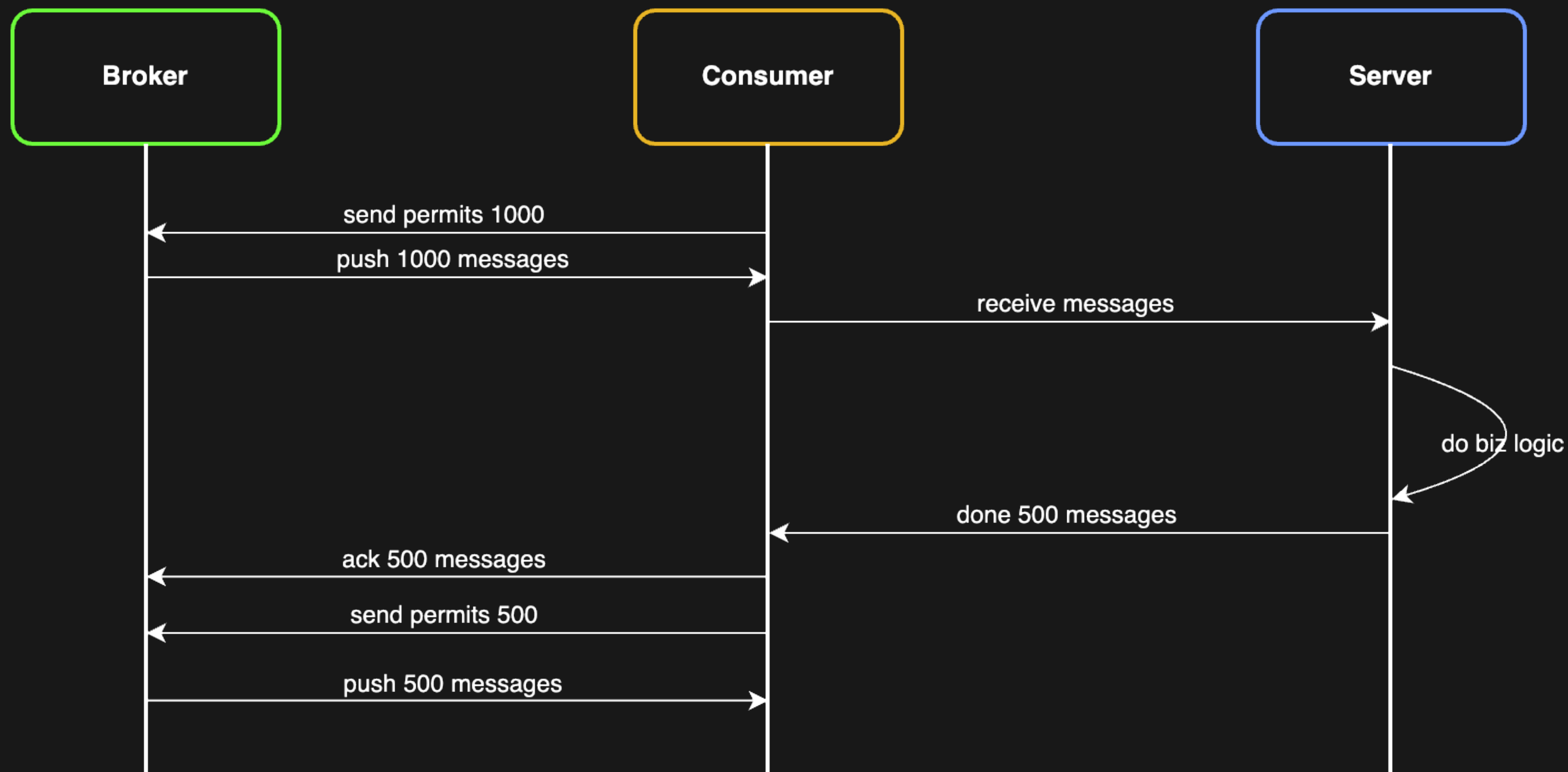
"activeConsumerName": "",
"consumers": [
  {
    "blockedConsumerOnUnackedMsgs": false,
    "availablePermits": 0,
    "unackedMessages": 1000,
    "msgRateOut": 0,
    "msgThroughputOut": 0,
    "msgRateRedeliver": 0,
    "consumerName": "ip-172-24-171-57.cn-northwest-1.compute.intel",
    "metadata": {
      "hostname": "ip-172-24-171-57.cn-northwest-1.compute.intel"
    }
  },
  {

```



Consumer Flow Control

“Flow” command: give permission to the broker to push messages



Consumer Flow Control

Understanding “Permits”

Best Practices #6

- FAQ: Why my consumer stops receiving new messages ?
 - Missing “ack” or too many “nack”
 - 0 permits
 - Messages have no chance to redeliver to other consumers
- Solution:
 - Enable RLQ/DLQ
 - Set VisibilityTimeout

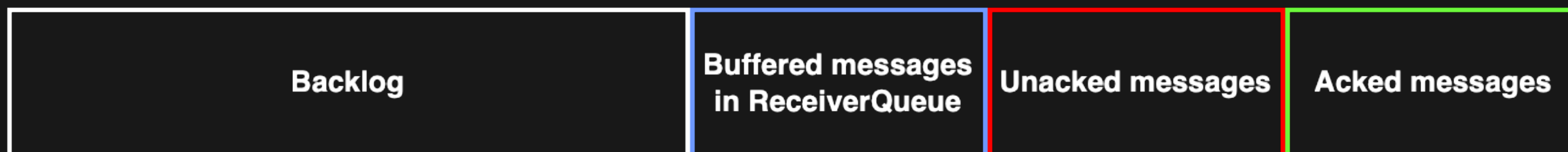
```

"activeConsumerName": "",
"consumers": [
  {
    "blockedConsumerOnUnackedMsgs": false,
    "availablePermits": 0,
    "unackedMessages": 1000,
    "msgRateOut": 0,
    "msgThroughputOut": 0,
    "msgRateRedeliver": 0,
    "consumerName": "ip-172-24-171-57.cn-northwest-1.compute.intel",
    "metadata": {
      "hostname": "ip-172-24-171-57.cn-northwest-1.compute.intel"
    }
  },
  {

```

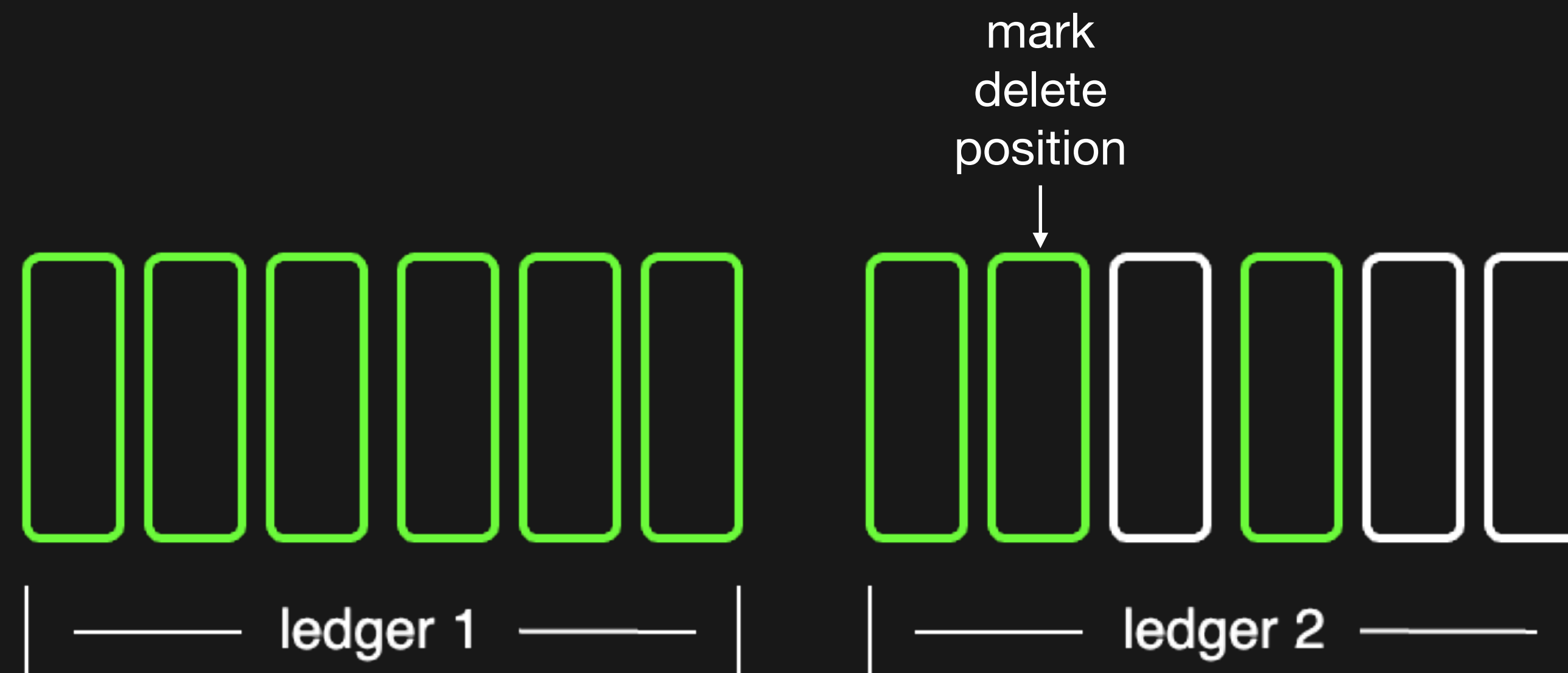
Messages Backlog

- Backlog: messages haven't been delivered yet
- Limitations:
 - MaxUnackedMessagesPerConsumer: 50000 (modifiable)
 - MaxUnackedMessagesPerSubscription: 200000 (modifiable)



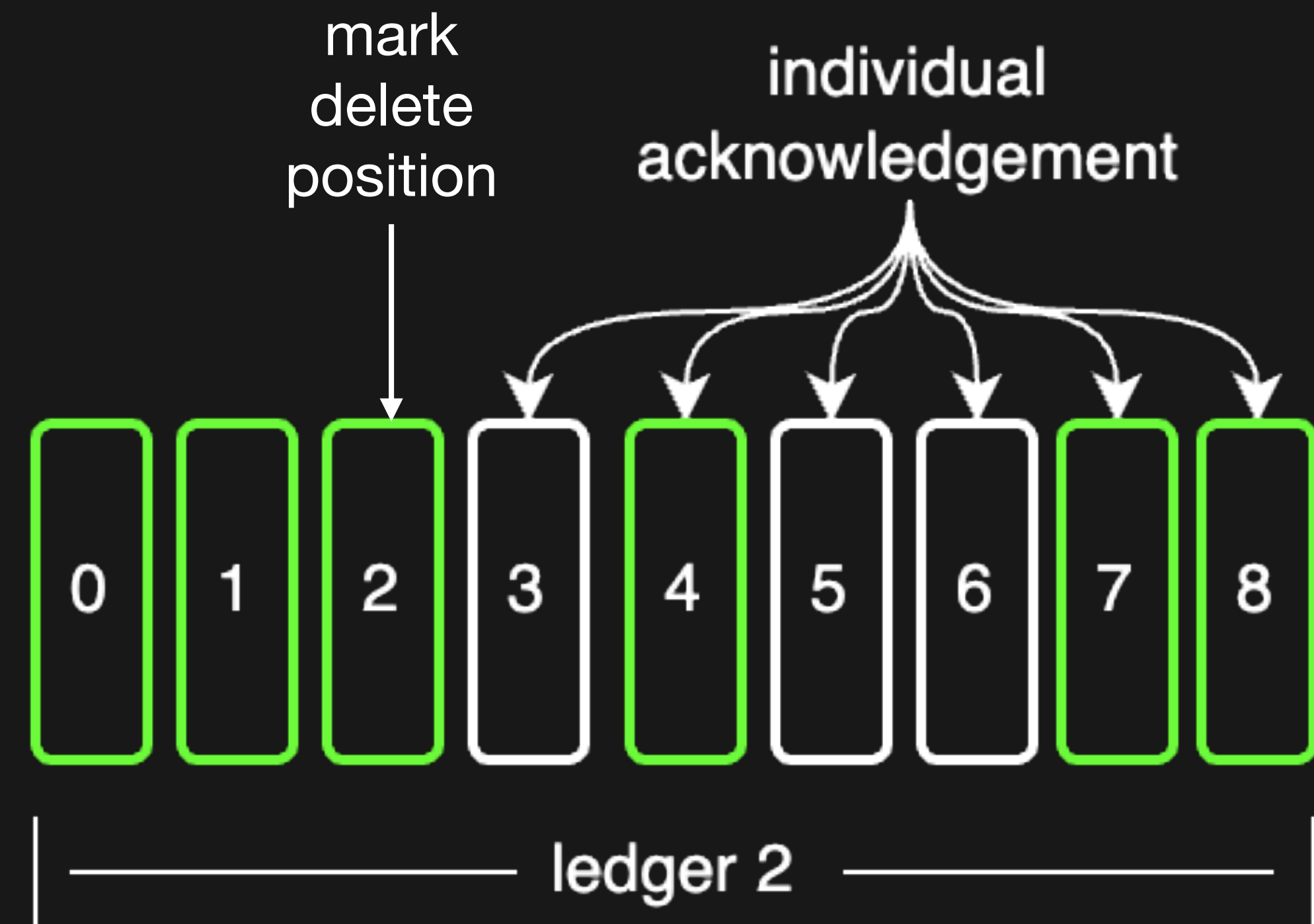
Unack Messages Limitation

- When one consumer goes down => too many redeliveries
- Costs of maintaining pending acknowledgements and non-continuous acknowledgement ranges



Acknowledgement State Management

- Mark delete position: (2,2)
- Cursor ledger (storage):
 - (2,4)
 - (2,7)
 - (2,8)



Acknowledgement State Management



- Limitations:
 - “managedLedgerMaxUnackedRangesToPersist” default is 10000
 - When reached, the info will only be tracked in memory
 - Messages will be redelivered after broker restart, the position will be the latest stored in cursor ledger (storage)

Best Practices #7

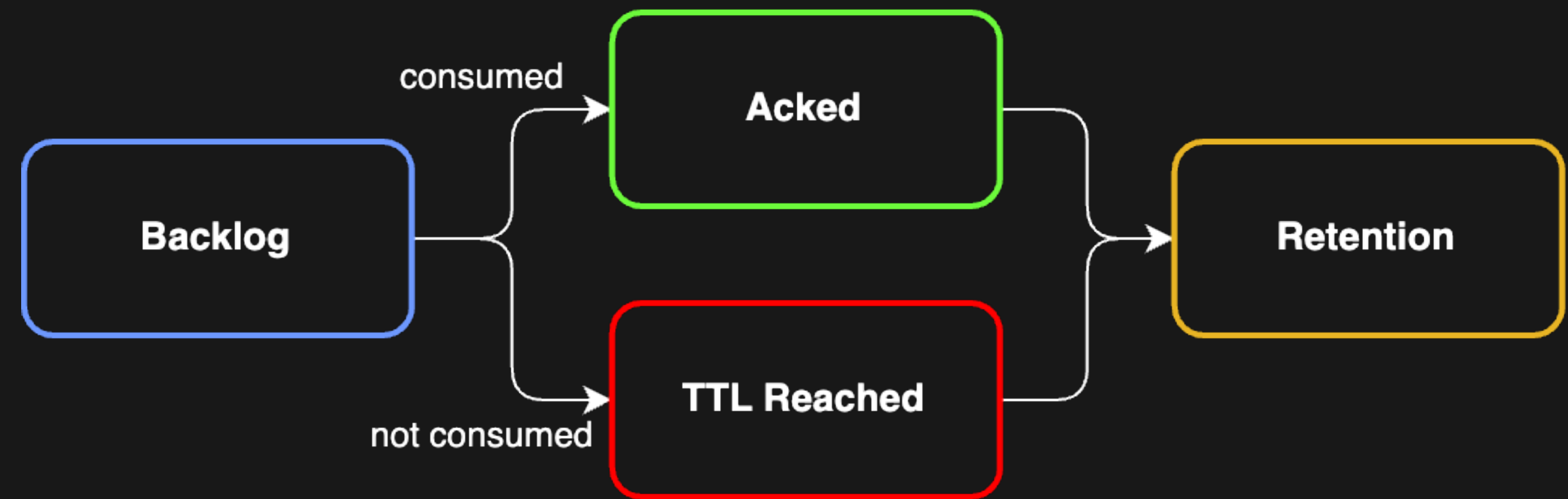
- To avoid leaving the “holes”
 - configure RLQ/DLQ
 - Make the best effort possible to “ack” the message instead of nack, since some failures you don’t even need to retry

Best Practices #8

- Pulsar uses at-least-once delivery semantics
 - Results to duplicated delivery
- Applications should implement idempotent or deduplication

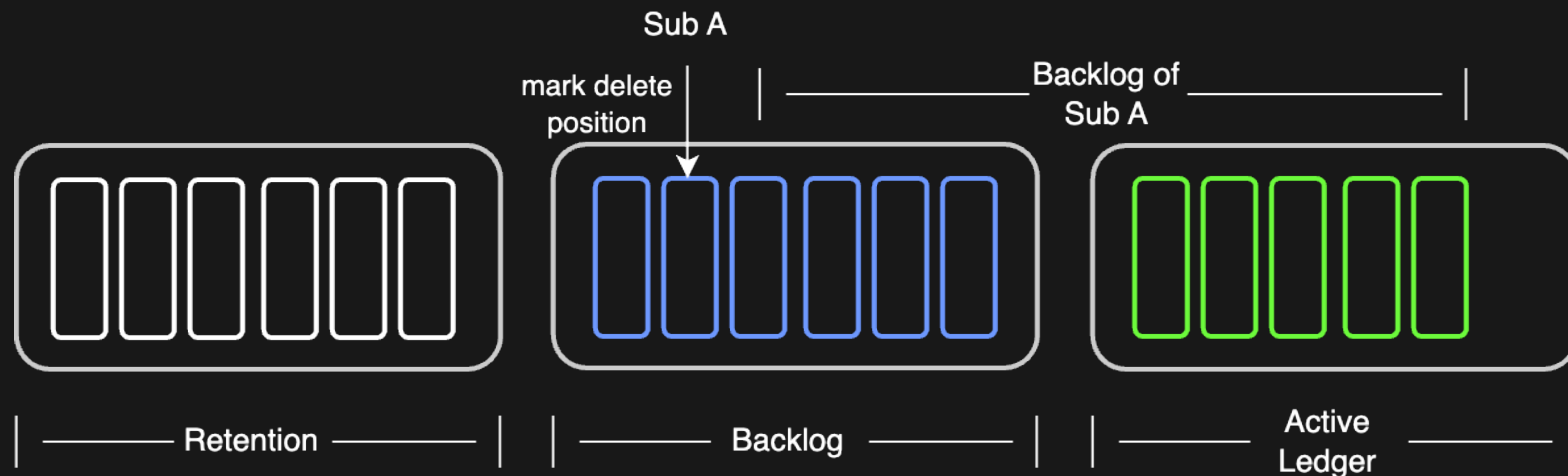
Message Lifecycle

- Data Retention
- Message TTL
- Backlog Quota



Message Lifecycle

- Data Retention
- Message TTL
- Backlog Quota



Data Retention

- Messages that have been acknowledged across all subscriptions
- When retention policy matched, will:
 - Permanently delete the data
 - Offload to tiered storage (s3)

Message Time-to-Live

- TTL only moves the “mark delete position”
- Messages that have reached the TTL will be moved to retention and not visible unless you reset the cursor

Backlog Quota

- Policies:
 - `producer_request_hold`
 - `producer_exception`
 - `consumer_backlog_eviction` (default)
- Limit:
 - Limit by size: 5 GB
 - Limit by time: 7 Days

Best Practices #9

- Always configure “Les Trois Mousquetaires”:
 - Retention Policy
 - Message TTL
 - Backlog Quota
- Use “pulsar_msg_backlog” to configure an alert to detect the subscriptions with issues
- Delete unused subscriptions to avoid keeping data around

```

apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
metadata:
  labels:
    prometheus: k8s
    thanos: k8s-app-thanos-2 # 指标当前并未用service monitor
    role: alert-rules
    name: prometheus-alerts-pulsar-metrics
    namespace: monitoring
spec:
  groups:
    - name: AppDLQMetricsBacklogCount
      rules:
        - alert: pulsar-pulsar_msg_backlog-count-10
          annotations:
            message: '[Pulsar业务] Backlog 数量大于10 pulsar_msg_backlog for 3 mins. [topic] {{ $labels.topic }}'
            请参考 Grafana: 手工Pulsar → Messaging Metrics 获取更多信息'
            summary: pulsar pulsar_msg_backlog count.
            expr: |
              pulsar_msg_backlog{topic =~"persistent://public/titan/event.*"} > 10
            for: 3m
            labels:
              severity: warning
              app: pagerduty

```

Setting Alarm

Example



←

→

↺

devops.planetmeican.com/#/pulsar/meican2/fan2-pulsar0/public/namespaces/titan/topics/event

☆

🌐

⚙️

📄

🗂️

👤

⋮

🏠 Home

🧩 Applications

📈 Observability ▾

💰 Cost Analysis

🚢 Pulsar ▾

📊 Log Pattern

📋 RDS Audit

🔔 Version: v0.4.2

☰

Home / Pulsar / Topics / Topic

🚀

▼

🔄

persistent://public/titan/event

Overview

Configuration

Partitions

Name	In Rate ⬆️	In Throughput ⬆️	Out Rate ⬆️	Out Throughput ⬆️	Storage Size ⬆️	Backlog Size ⬆️	Actions
persistent://public/titan/event-partition-0	0.53/s	987.0/s	1.07/s	2.0k/s	4.1G	0	Messages

Subscriptions

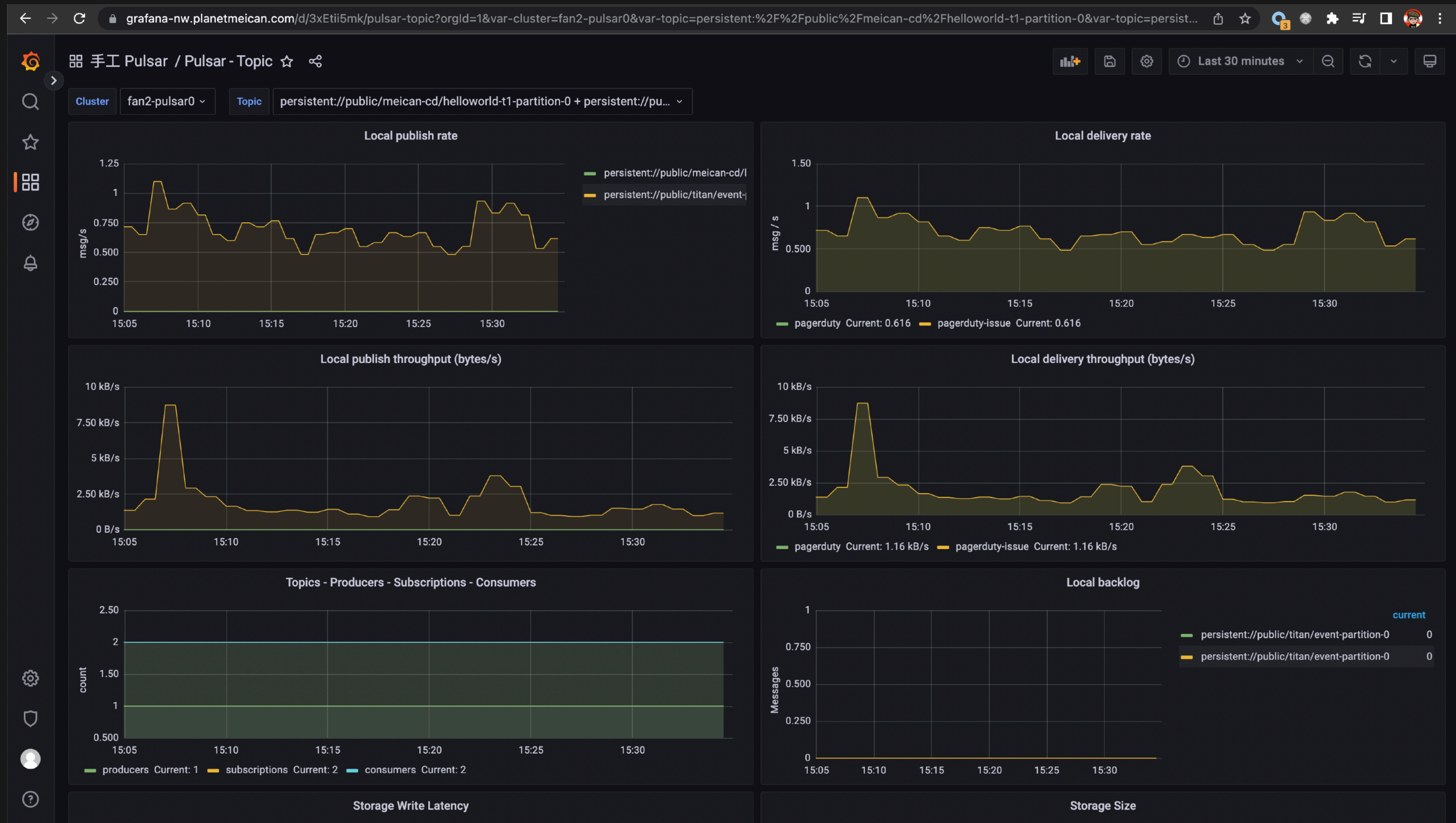
name	Out Rate ⬆️	Out Throughput ⬆️	Backlog w/o Delayed ⬆️	Backlog w/ Delayed ⬆️	Actions
> pagerduty	0.53/s	987.0/s	0	0	Backlog Operation
> pagerduty-issue	0.53/s	987.0/s	0	0	Backlog Operation

Producers

ID	Name	Mode	Address	In Rate ⬆️	In Throughput ⬆️	Avg. Message Size ⬆️	Since
1	pagerduty-controller-manager-859f58bdc8-zpw xj	Shared	/10.120.156.94:46938	0.53/s	987.0/s	1.9k	2023-05-05T09:34:45.705527Z

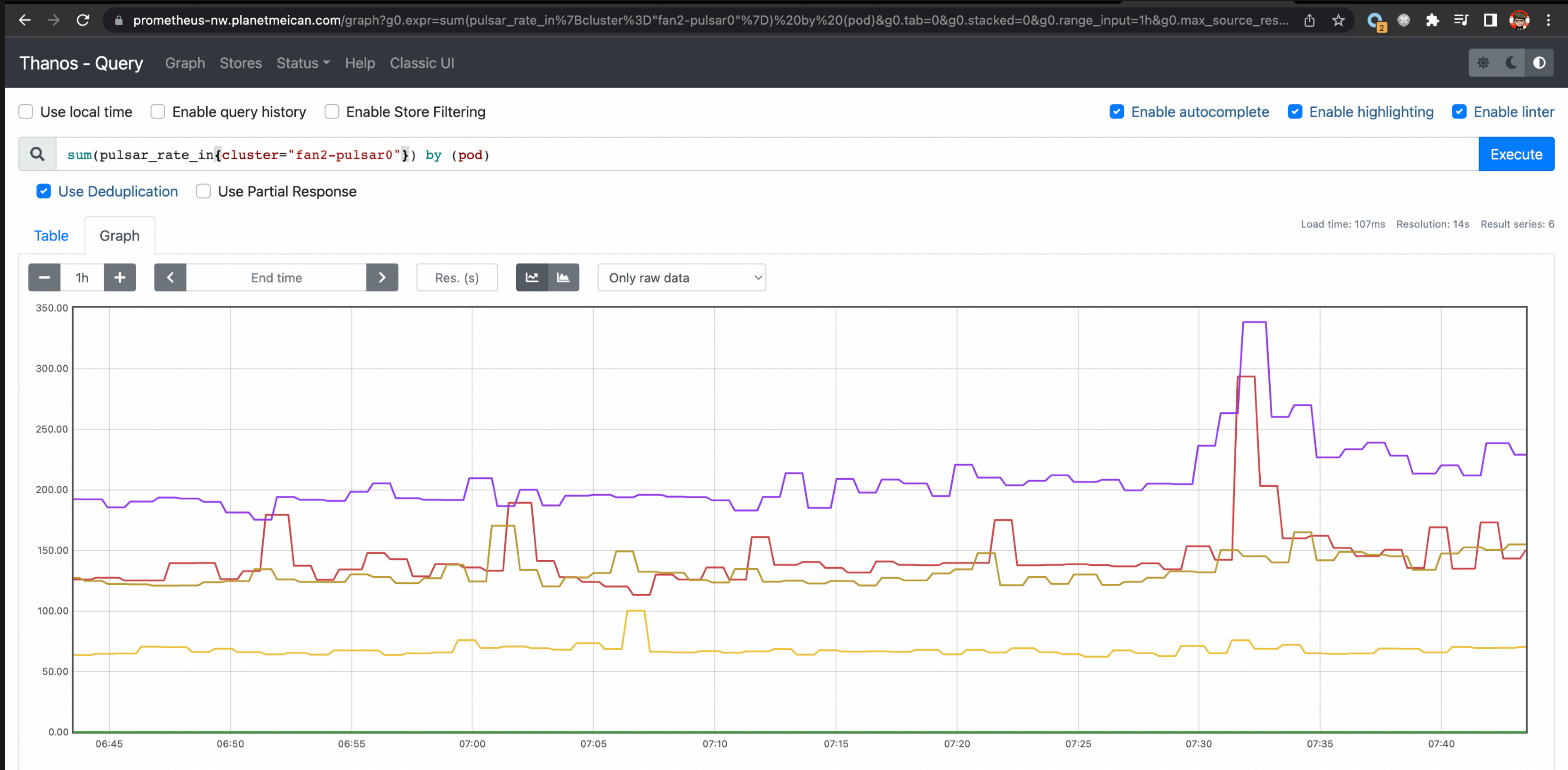
Tools

DevOps Platform



Tools

Grafana Dashboard



Tools

Prometheus

Fin.

Questions ?